



Taller de Comandos Linux

Emilio Albán, Ariel Montufar, Samuel Churaco, Karen Suarez

2026-06-19

1 Objetivos

- Conocer el funcionamiento del terminal de *Linux*
- Aprender a manejar comandos del terminal de *Linux*

2 Instrucciones

Este taller/tutorial de comandos de Linux tiene por objetivo ejercitar en la aplicación de comandos estudiados en clase. Para esto siga las Instrucciones indicadas en cada uno de los numerales. Ejecute los ejercicios llenando los comandos adecuados dentro de los bloques de código de Emacs Org. Los resultados de la ejecución se obtienen haciendo `C-c C-c`

Para todos los ejercicios se sugiere usar directamente un distro de *Linux* o el *WSL* configurado para la clase de *Arquitectura de Computadores*. Ejecute los comandos en la terminal de Linux. Copie los comandos usados a este archivo.

Si se ha utilizado IA para consultar temas referentes al desarrollo de código en el punto 4, adjunte el archivo `.md` con el reporte de los diferentes prompts y respuestas dadas por la IA.

2.1 Ejecución desde Emacs

Si desea ejecutar los comandos desde Emacs debe observar que los comandos del `shell` de Linux se ejecutarán desde la ubicación del archivo `.org`. Por tanto, controle la ubicación de `home` en los bloques de código. Para ejecutar los bloques de código ejecute: `C-c C-c`.

3 Comandos de Linux

Para este taller descargaremos los datos del *Fashion-Mnist*, un dataset popular para la prueba de algoritmos de machine learning. Al final, el dataset quedará organizado para ser aprovechado para entrenar un modelo.

1. Utilice `cd` para localizarse en la posición de `home` de su usuario y verifique ejecutando el comando `pwd`

```
cd
pwd
```

```
/home/arielmontufar
```

1. Cree tres directorios denominados `FashionData`, `Train` y `Test`. Verifique usando el comando `ls`

```
ls -ld ~/FashionData ~/Train ~/Test
```

```
drwxr-xr-x 2 arielmontufar arielmontufar 4096 Jun 25 21:59 /home/arielmontufar/FashionData
drwxr-xr-x 2 arielmontufar arielmontufar 4096 Jun 25 22:13 /home/arielmontufar/Test
drwxr-xr-x 2 arielmontufar arielmontufar 4096 Jun 25 21:59 /home/arielmontufar/Train
```

1. Investigue sobre el comando `wget` y utilícelo para descargar en `~/FashionData` los datos de train y test del `/Fashion-Mnist` desde el siguiente enlace: `fashion>-mnist`

(a) Imágenes de entrenamiento:

```
http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-images-idx3-ubyte.
gz
```

(b) Etiquetas de entrenamiento:

```
http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-labels-idx1-ubyte.
gz
```

(c) Imágenes de prueba:

```
http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-images-idx3-ubyte.
gz
```

(d) Etiquetas de prueba:

```
http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-labels-idx1-ubyte.
gz
```

```
cd ~/FashionData
wget -c http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-images-idx3-ubyte.gz
wget -c http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-labels-idx1-ubyte.gz
wget -c http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-images-idx3-ubyte.gz
wget -c http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-labels-idx1-ubyte.gz

ls -lh
```

```
total 30M
-rw-r--r-- 1 arielmontufar arielmontufar 4.3M Aug 31 2017 t10k-images-idx3-ubyte.gz
-rw-r--r-- 1 arielmontufar arielmontufar 5.1K Aug 31 2017 t10k-labels-idx1-ubyte.gz
-rw-r--r-- 1 arielmontufar arielmontufar 26M Aug 31 2017 train-images-idx3-ubyte.gz
-rw-r--r-- 1 arielmontufar arielmontufar 29K Aug 31 2017 train-labels-idx1-ubyte.gz
```

2. Utilice el comando `mv` de manera recursiva para pasar los datos de entrenamiento a la carpeta `~/Train`

```
mv ~/FashionData/*train* ~/Train/
ls -lh ~/Train/
```

```
total 51M
-rw-r--r-- 1 arielmontufar arielmontufar 26M Aug 31 2017 train-images-idx3-ubyte.gz
-rw-r--r-- 1 arielmontufar arielmontufar 26M Aug 31 2017 train-images-idx3-ubyte.gz.1
-rw-r--r-- 1 arielmontufar arielmontufar 29K Aug 31 2017 train-labels-idx1-ubyte.gz
```

3. Utilice el comando `mv` de manera recursiva para pasar los datos de prueba a la carpeta `~/Test`

```
mv ~/FashionData/t10k* ~/Test/
ls -lh ~/Test/
```

```
total 4.3M
-rw-r--r-- 1 arielmontufar arielmontufar 4.3M Aug 31 2017 t10k-images-idx3-ubyte.gz
-rw-r--r-- 1 arielmontufar arielmontufar 5.1K Aug 31 2017 t10k-labels-idx1-ubyte.gz
```

4. Verifique que la carpeta FashionData está vacía

```
ls -lh ~/FashionData/
```

```
total 0
```

5. Los archivos anteriores están comprimidos. Investigue como descomprimir usando el comando **gunzip**. Si es necesario realice la instalación del comando. Descomprima los archivos en sus respectivas carpetas. Apunte el comando utilizado para la descompresión.

```
gunzip -d ~/Train/*.gz
gunzip -d ~/Test/*.gz
ls -lh ~/Train/
ls -lh ~/Test/
```

```
total 45M
-rw-r--r-- 1 arielmontufar arielmontufar 45M Aug 31 2017 train-images-idx3-ubyte
-rw-r--r-- 1 arielmontufar arielmontufar 59K Aug 31 2017 train-labels-idx1-ubyte
total 7.5M
-rw-r--r-- 1 arielmontufar arielmontufar 7.5M Aug 31 2017 t10k-images-idx3-ubyte
-rw-r--r-- 1 arielmontufar arielmontufar 9.8K Aug 31 2017 t10k-labels-idx1-ubyte
```

6. Mueva recursivamente las carpetas ~/Train y ~/Test dentro de ~/FashionData

```
mv ~/Train ~/FashionData/
mv ~/Test ~/FashionData/
ls -R ~/FashionData/
```

```
/home/arielmontufar/FashionData/:
Test
Train
```

```
/home/arielmontufar/FashionData/Test:
t10k-images-idx3-ubyte
t10k-labels-idx1-ubyte
```

```
/home/arielmontufar/FashionData/Train:
train-images-idx3-ubyte
train-labels-idx1-ubyte
```

7. Verifique la estructura de la carpeta /FashionData usando el comando **tree**

```
tree ~/FashionData/
```

```
/home/arielmontufar/FashionData/
Test
  t10k-images-idx3-ubyte
  t10k-labels-idx1-ubyte
```

```

Train
  train-images-idx3-ubyte
  train-labels-idx1-ubyte

```

3 directories, 4 files

4 Problema

Se desea realizar un registro climatológico de la ciudad de Quito. Para esto, escriba un script de Python/Java que permita obtener datos climatológicos desde el API de openweathermap. Considere la latitud y la longitud de Quito como (-0.2299, -78.5249), respectivamente. Los resultados obtenidos de la consulta al API se escriben en un archivo *clima-quito-hoy.csv*. Cada ejecución del script debe almacenar nuevos datos en el archivo. Investigue sobre **crontab** y utilícelo para obtener datos del API de *openweathermap* cada 5 minutos mediante la ejecución de un archivo ejecutable denominado *get-weather.sh*. Verifique los resultados. Todas las operaciones se realizan en Linux o en el WSL. Las etapas del problema se subdividen en:

1. Crear su API gratuito en openweathermap API: f0a67d31e3a1532c0e0404b8b5b2525f
2. Escribir un script en Python/Java que realice la consulta al API y escriba los resultados en *clima-quito-hoy.csv*
 - (a) Primero se creó el archivo clima-quito-hoy.csv en la terminal con touch.
 - (b) Desarrollo del Script: Importante: Se utilizó la librería externa JSON, por lo tanto se instaló el json-20231013.jar, se lo compiló y ejecutó con el .jar ya mencionado (ya compilado y ejecutando en la terminal de esta manera):

```

javac -cp ./json-20231013.jar RegistroClimaticoQuito.java
java -cp ./json-20231013.jar RegistroClimaticoQuito

```

```

import java.net.URL;
import java.net.HttpURLConnection;
import java.io.*;
import org.json.*;
import java.time.LocalDateTime;

```

```

public class RegistroClimaticoQuito{

```

```

    //API KEY generado de OpenWeatherMap
    private static final String API_KEY = "f0a67d31e3a1532c0e0404b8b5b2525f";
    //Latitud y Longitud de Quito
    private static final double LATITUD = -0.2299;
    private static final double LONGITUD = -78.5249;

```

```

    //Nombre del archivo en donde se escribirán los resultados
    private static final String ARCHIVO = "clima-quito-hoy.csv";

```

```

    public static void main(String[] args) {
try{ //Controlar errores y que el programa no quede sin finalizar
    String url = construirURL(); //Se construye el URL Del API
    String respuesta = consultarAPI(url); //Se consulta el API y se obtiene la respuesta JSON Como text
    JSONObject datos = convertirJSON(respuesta); //Se convierte de String-JSON a objeto JSON
    guardarCSV(datos); //Se guardan los datos en clima-quito-hoy.csv
}
}

```

```

        System.out.println("Datos guardados correctamente.");

    }catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}

public static String construirURL(){
    // se arma la URL con latitud, longitud de Quito, unidades(C°) y API key
    String url =
        "https://api.openweathermap.org/data/2.5/weather"
        + "?lat=" + LATITUD
        + "&lon=" + LONGITUD
        + "&units=metric"
        + "&lang=es"
        + "&appid=" + API_KEY;

    // se retorna la URL completa
    return url;
}

public static String consultarAPI(String urlString) throws Exception {

    URL url = new URL(urlString); // crear objeto URL apartir del string URL
    HttpURLConnection con = (HttpURLConnection) url.openConnection(); // abrir conexión HTTP
    con.setRequestMethod("GET"); // definir método GET

    // Se lee respuesta del servidor
    BufferedReader reader = new BufferedReader(new InputStreamReader(con.getInputStream()));

    StringBuilder respuesta = new StringBuilder(); // almacenar respuesta completa
    String linea; // variable para lectura línea por línea
    // leer todo el contenido de la respuesta
    while ((linea = reader.readLine()) != null) {
        respuesta.append(linea);
    }

    reader.close(); // cerrar lector
    return respuesta.toString(); // devolver JSON como texto
}

public static JSONObject convertirJSON(String respuesta) {
    // convertir string JSON a objeto JSON para los datos al .csv
    JSONObject obj = new JSONObject(respuesta);
    // retornar objeto JSON
    return obj;
}

public static void guardarCSV(JSONObject obj) throws Exception {

    // verifica si el archivo ya existe cuando se coloque el encabezado
    File file = new File(ARCHIVO);

```

```

        FileWriter csvWriter = new FileWriter(ARCHIVO, true); //se abre el archivo en modo append (no b

// si el archivo está vacío/nuevo,se escribe el encabezado
if (file.length() == 0) {
    csvWriter.append("fecha,ciudad,temperatura,humedad,presion,descripcion\n");
}

String fechaHora = LocalDateTime.now().toString(); //Fecha de guardado
//SE EXTRAE DATOS DEL JSON:
String ciudad = obj.getString("name"); //el nombre de la ciudad
JSONObject main = obj.getJSONObject("main"); // el conjunto main del JSON de lo retornado a la l
double temp = main.getDouble("temp"); // temperatura del conjunto main
int humedad = main.getInt("humidity");// humedad del conjunto main
int presion = main.getInt("pressure");// presión del conjunto main
JSONArray weather = obj.getJSONArray("weather");//el arreglo weather del JSON de lo retornado a
String descripcion = weather.getJSONObject(0).getString("description");//descripción del clima (

// ESCRIBIR EN CSV usando coma como separador
csvWriter.append(fechaHora + "," +
                ciudad + "," +
                temp + "," +
                humedad + "," +
                presion + "," +
                descripcion + "\n");
csvWriter.close();//se cierra el archivo
}
}

```

1. Desarrollar un ejecutable *get-weather.sh* para ejecutar el programa Python/Java. Nota: Se creó el .sh en la terminal con touch y su contenido es el siguiente:

```

#!/bin/bash
# 1. Ubica el script en el directorio del proyecto
cd "$(dirname "$0")"

# 2. Se compila el archivo Java
javac -cp .:json-20231013.jar RegistroClimaticoQuito.java

# 3. Ejecuta el programa Java usando el classpath especificado
java -cp .:json-20231013.jar RegistroClimaticoQuito

# 4. Dar permisos de ejecución pero directamente desde la terminal con: chmod +x get-weather.sh

```

1. Configurar Crontab para la adquisición de datos. Escriba el comando configurado.

```

crontab -e
# Se agrega directamente a la linea del crontab:
# */5 * * * * $HOME/G1ArqComp/G1ArqComp/Talleres/get-weather.sh
# significa cada 5 minutos el cron ejecutará el programa.

```